CMP720 Embedded System Design

WEEK 10

2026 Spring

https://piazza.com/hacettepe.edu.tr/spring2026/cmp720
https://github.com/sepidi/hacettepe-cmp720-spring-2026

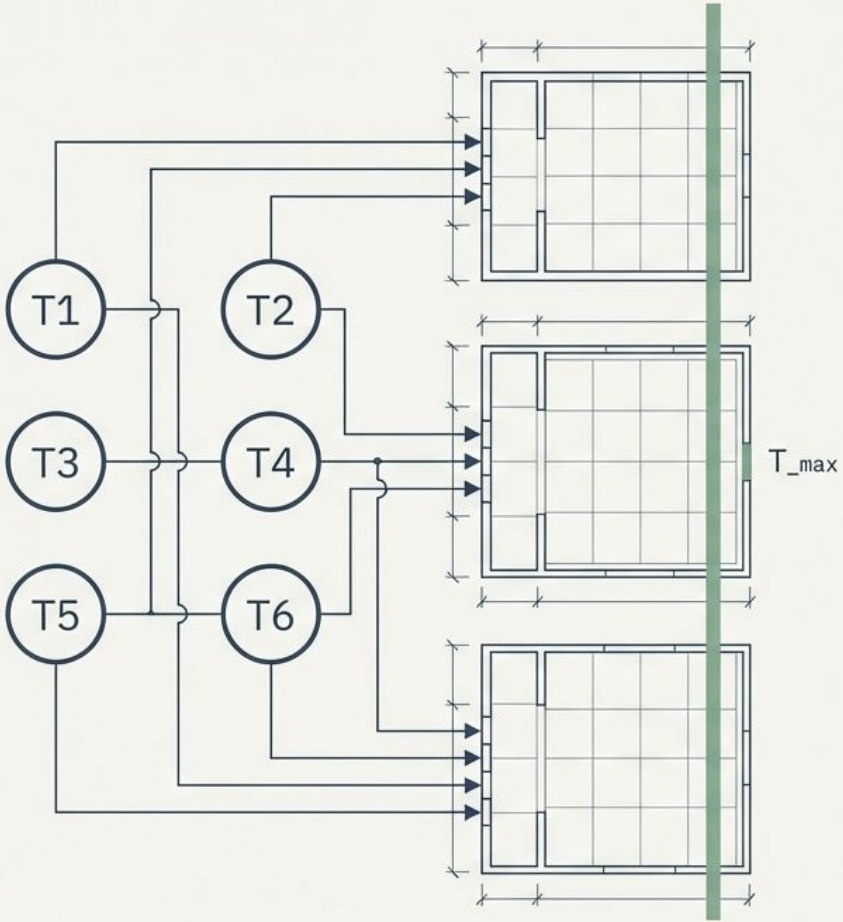
THE PLAN FOR TODAY

Weeks 8+ Topics: More Optimization

Makespan Optimization via ILP: Two Formulations

Metaheuristic Approach: Simulated Annealing

How variable choice shapes formulation complexity



Makespan Optimization via ILP

Formulating the Heterogeneous Embedded Task Scheduler

Why fewer variables don't always mean a simpler model.

NotebookLM

Problem Description

Scenario

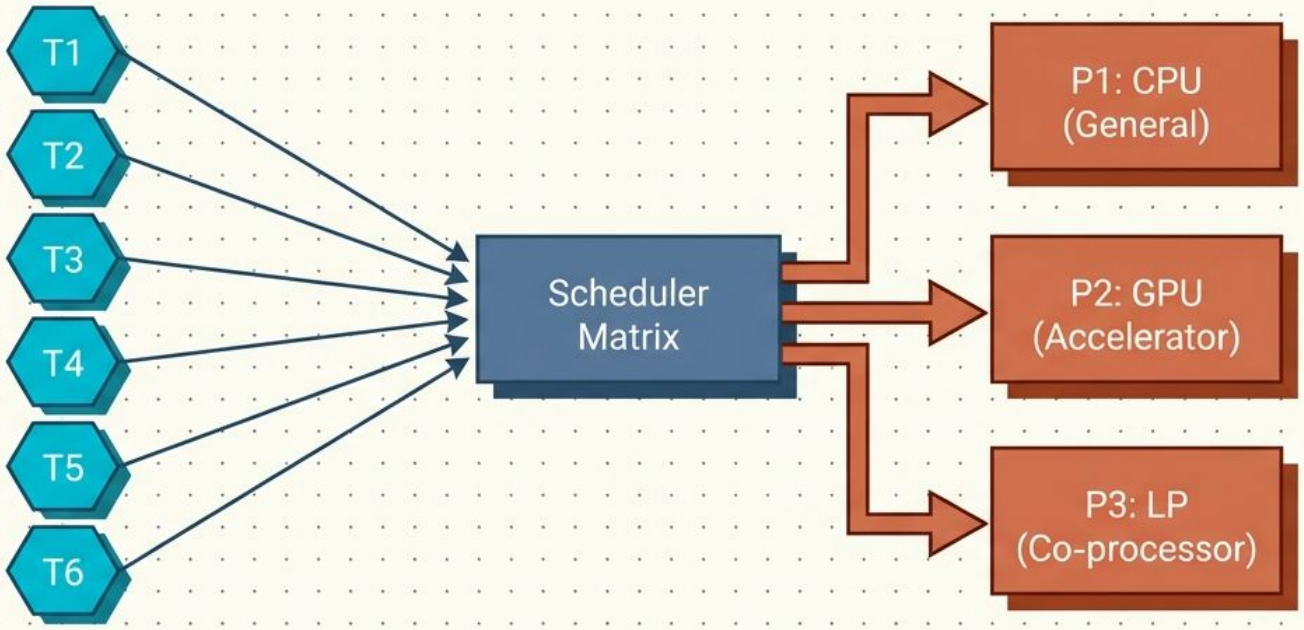
Heterogeneous embedded platform
with 3 processing elements:

P1 — General-purpose CPU
P2 — GPU accelerator
P3 — Low-power co-processor

6 independent tasks to execute. Same task takes different time on each processor.

GOAL

Assign every task to exactly one processor to minimize the makespan



The Goal

Assign exactly 6 independent tasks to 3 processing elements to minimize the total completion time.

The Catch

The platform is heterogeneous. Different hardware architectures mean the exact same task takes a drastically different amount of time depending on where it is assigned.

NotebookLM

Key Terminology

Makespan

Total completion time = maximum load across all processors. Minimizing makespan means balancing the workload.

Load

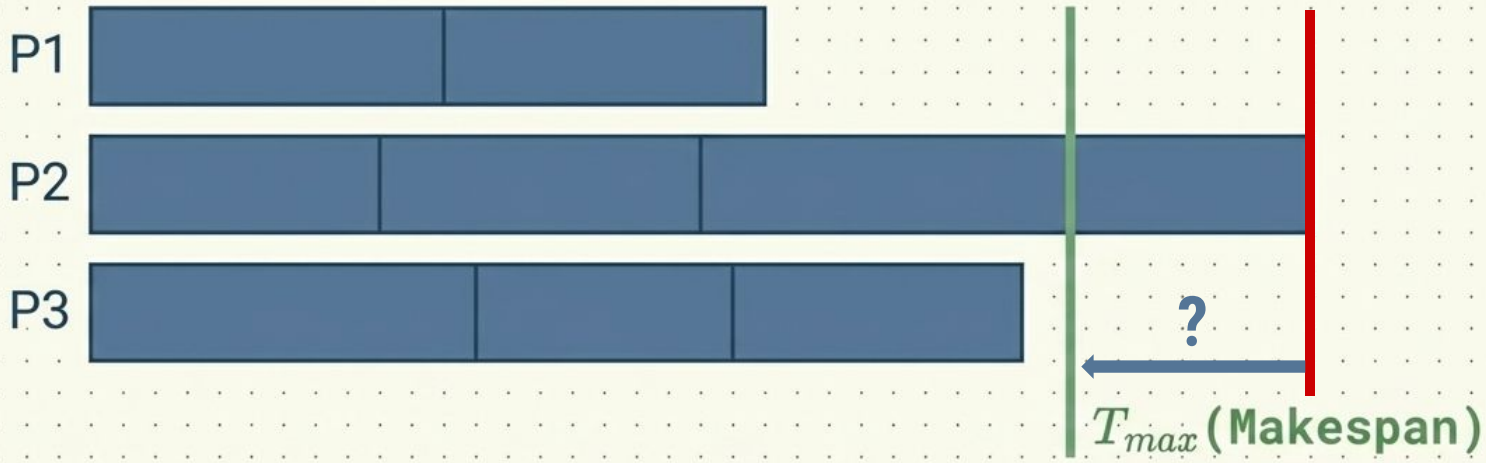
Sum of execution times of all tasks assigned to a particular processor.

Assignment Problem

Each item must be assigned to exactly one resource — no splitting, no leaving unassigned.

Minimax Objective

"**Minimize the maximum of** …" — not directly linear, requires a linearization trick.



Makespan

The completion time of the last processor to finish. Minimizing this strictly forces the scheduler to balance the overall workload.

Load

The sum of execution times for all tasks assigned to one specific processor.

Assignment Problem

The strict 1-to-1 mapping rule governing the system—each task must go to exactly one resource. It cannot be split or skipped.

NotebookLM

Problem Data: Execution Time Matrix

$t(i,j)$ = execution time of task i on processor j (time units)

	P1 (CPU)	P2 (GPU)	P3 (LP)
T1	3	7	5
T2	8	2	6
T3	6	3	4
T4	4	5	9
T5	5	8	3
T6	7	4	8

Note how T2 runs 4× faster on GPU (2) vs CPU (8) — but T4 prefers the CPU. No single processor is best for all tasks.

Problem Data: Memory Constraints

Task Memory Footprint

mem(i) - memory footprint of Task i

T1	T2	T3	T4	T5	T6
2	3	2	4	1	3

Processor Memory Capacity

P1 (CPU)	P2 (GPU)	P3 (LP)
8	7	6

cap(j) - memory capacity of processor j

Feasibility Check

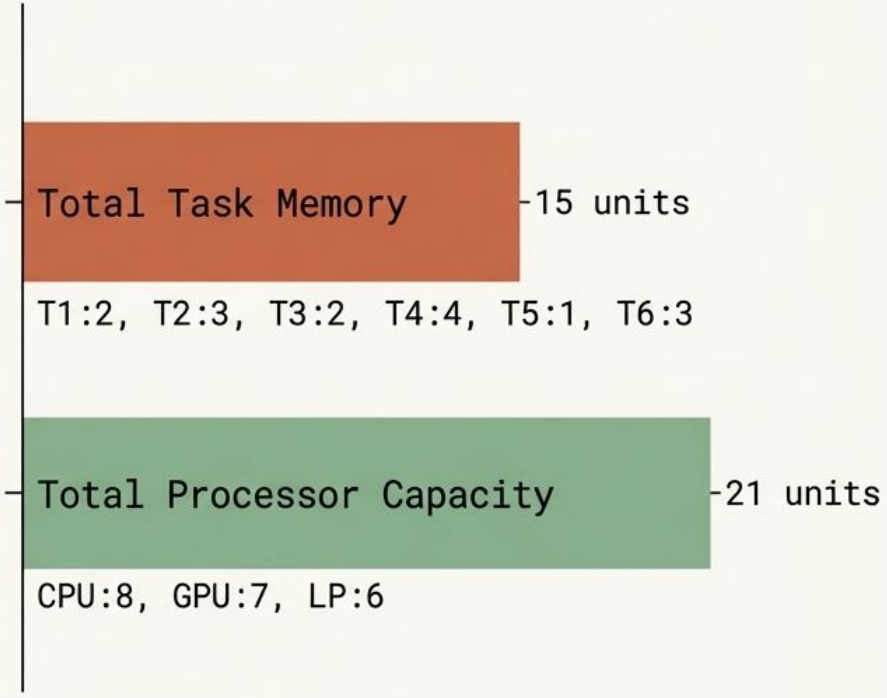
Total task memory = 15
Total capacity = 21

Globally feasible, but not every assignment respects individual processor limits.

The Data Landscape: 6 Tasks, 3 Processors

	P1 (CPU)	P2 (GPU)	P3 (LP)
T1	3	7	5
T2	8	2	6
T3	6	3	4
T4	4	5	9
T5	5	8	3
T6	7	4	8

Time execution matrix



Insight: The system is globally feasible (15 footprint < 21 capacity), but local assignments must be carefully managed.

NotebookLM

Thinking About the Formulation

Before writing any math, answer these questions:

Q1 **What are you minimizing?**

Can you write it as a single linear expression? If not, what trick might help?

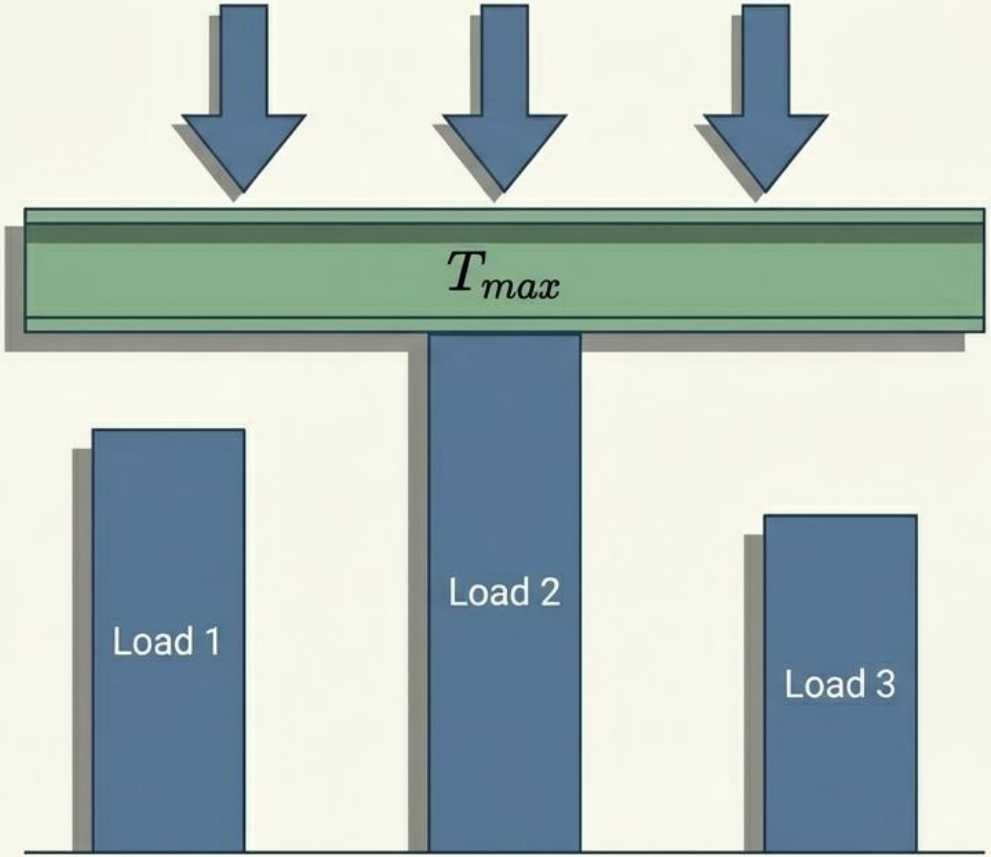
Q2 **What must be true about every valid assignment?**

Can a task be split across processors? Can a task be left unassigned?

Q3 **How do you express "the load on processor j"?**

This determines everything — the objective, the constraints, the variable choice.

Linearizing the Objective: The Minimax Trick



The Problem

"Minimize the maximum load" is inherently non-linear. A linear solver cannot directly compute a max() function in its objective.

The Trick

Introduce a continuous auxiliary variable, T_{max}.

The Execution

Replace the objective with simply 'minimize T_{max}'. Then, add strict constraints forcing T_{max} to be greater than or equal to every individual processor's load

As the solver pushes T_{max} downward, it inevitably flattens the highest load.

NotebookLM

The Minimax Linearization Trick

"minimize max(...)" is NOT linear!

1. Introduce an auxiliary variable `T_max`
2. Set objective: `minimize T_max`
3. Add constraints: `T_max ≥ Load(j) for each j`

> **Why it works:** The solver pushes T_max down as far as possible. The constraints force T_max ≥ each load, so at optimum T_max = max load.

Model A: The Binary Blueprint

	P1 (CPU)	P2 (GPU)	P3 (LP)
T1	0	1	0
T2	1	0	0
T3	0	0	1
T4	0	1	0
T5	1	0	0
T6	0	0	1

The Variable

x(i,j) in {0,1}

The Meaning

1 means "Task i runs on Processor j".
0 means it does not.

Total Scope

19 total variables (18 binary assignment variables + 1 continuous variable for T_max).

Philosophy

Create a strict boolean grid. It requires more variables upfront, but mathematically, it speaks the exact language of an assignment problem.

NotebookLM

MODEL A

Binary ILP — Decision Variables

$x(i,j) \in \{0, 1\}$

$x(i,j) = 1$ if task i is assigned to processor j

$x(i,j) = 0$ otherwise

for $i = 1,\ldots,6$ and $j = 1,\ldots,3$

$T_max \geq 0$

Auxiliary continuous variable — the makespan

Variable Count

18

binary (6 × 3)

+ 1

continuous (T_max)

= 19 total

MODEL A

Binary ILP — Constraints

i → task index
j → core index

Objective: minimize T_max

0	1	0
1	0	0
0	0	1
0	1	0
1	0	0
0	0	1

sum over each row must be = 1

C1 — Assignment

$\forall i: \ \Sigma \square \ x(i,j) = 1$

Each task goes to exactly one processor

6 constraints

C2 — Makespan Linking

$\forall j: \ T_max \geq \Sigma_i \ t(i,j) \cdot x(i,j)$

where $\Sigma_i \ t(i,j) \cdot x(i,j) = Load(j)$ **why?**

T_max must be ≥ load on every processor

3 constraints

C3 — Memory Capacity

$\forall j: \ \Sigma_i \ mem(i) \cdot x(i,j) \leq cap(j)$

Total memory on each processor must fit

3 constraints

Total: 12 constraints

MODEL A

Binary ILP — Summary

Aspect	Count
Binary variables	18 (6×3)
Continuous variables	1
Total decision variables:	**19**
Assignment constraints	6
Makespan constraints	3
Memory constraints	3
Total constraints:	**12**

Strengths

- Clean, direct formulation
- Every constraint is a straightforward linear expression
- Solvers exploit assignment matrix structure
- Tight LP relaxation → fast solving

MODEL B

Integer ILP — The Idea

"What if we used fewer variables?" `y(i) ∈ {1, ..., m}`

y(i) = processor index for task i

e.g. y(3) = 2 means "T3 runs on P2"

Only 6 integer variables + 1 continuous (T_max) = 7 total!

7

variables

vs 19 in Model A

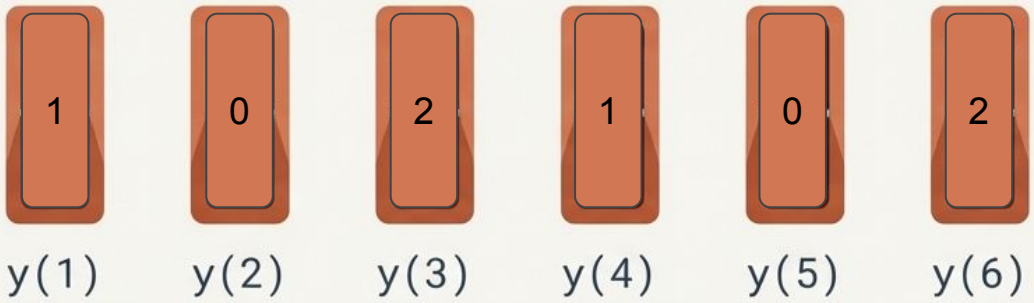
?

But wait...

How do you compute "the load on processor j"?

You need to know which tasks have y(i) = j. Can a linear constraint express this?

Model B: The Integer Illusion



The Variable

```
y(i) in {1, 2, 3}
```

A single integer representing the exact index of the processor to which task i is assigned.

Perceived Scope

Only 7 total variables (6 integer variables + 1 continuous variable for T_max).

The Appeal

At first glance, this is mathematically elegant. It dramatically reduces the variable count and perfectly matches how a human naturally thinks about assigning an item to a bucket.

NotebookLM

MODEL B

The Problem — Expressing Load

We need:

$$\text{Load}(j) = \Sigma_i \; t(i,j) \cdot [y(i) = j]$$

[y(i) = j] is the Iverson bracket: 1 if true, 0 if false

This is NOT linear!

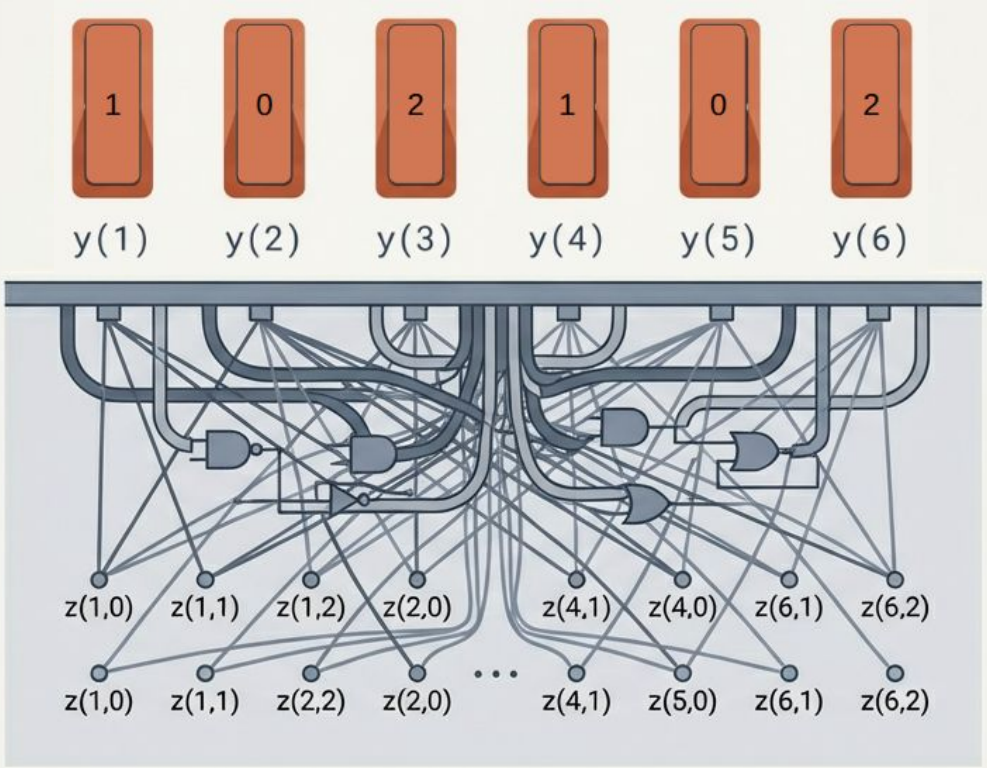
The indicator [y(i) = j] is a nonlinear function of y(i).

We cannot write $T_max \geq \Sigma_i \, t(i,j) \cdot [y(i) = j]$ in a linear program.

We need a way to linearize this relationship.

Solution → Introduce auxiliary binary variables

The Linearization Trap



The Breakdown

We need to calculate processor load. But we cannot write an equation that says "If **y(i)=j**, add time to load" in a linear solver. The indicator function **[y(i) = j]** is non-linear.

The Forced Fix

To make the math work, we are forced to introduce 18 hidden auxiliary binary variables **z(i,j)** where **z(i,j) = 1** if and only if **y(i) = j**.

The Cost

We must now write complex linking constraints just to force the integers and the hidden binaries to agree with each other.

NotebookLM

MODEL B

The Fix — Auxiliary Binary Variables

Introduce $z(i,j) \in \{0,1\}$ where $z(i,j) = 1$ iff $y(i) = j$

Linking Constraints

L1 — Sum constraint

$$\forall i : \sum_{j} z(i,j) = 1$$

6 constraints

L2 — Value constraint

$$\forall i : y(i) = \sum_{j} j \cdot z(i,j)$$

6 constraints

Now the makespan and memory constraints use z(i,j) — identical to Model A!

$$\forall j : T_max \geq \sum_i t(i,j) \cdot z(i,j)$$

3 makespan constraints

$$\forall j : \sum_i mem(i) \cdot z(i,j) \leq cap(j)$$

3 memory constraints

MODEL B

Integer ILP — Summary

Aspect	Count
Integer variables (y)	6
Auxiliary binary variables (z)	18 (6×3)
Continuous variables	1
Total decision variables:	25
Linking constraints (L1 + L2)	12
Makespan constraints	3
Memory constraints	3
Total constraints:	**18**

Model A

19 vars · 12 constr

→

Model B

25 vars · 18 constr

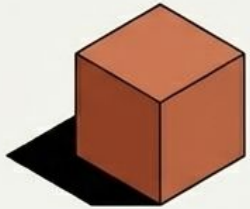
THE IRONY

Started with 6 integer variables hoping for a simpler model…

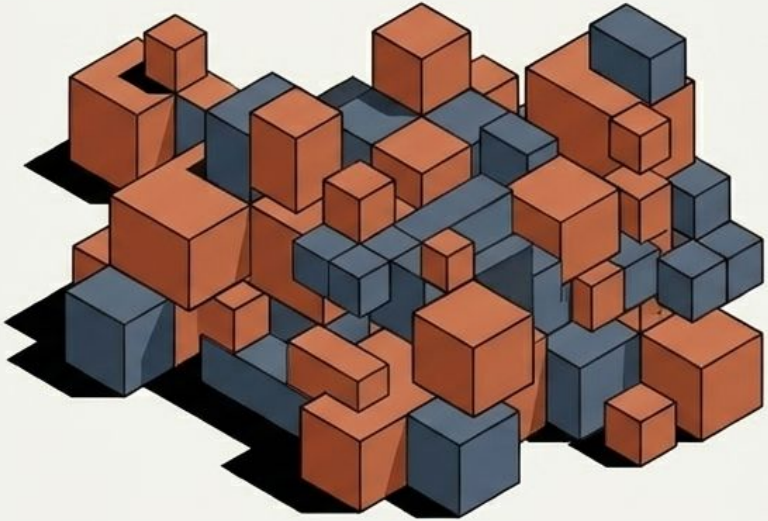
Had to add 18 auxiliary binary variables + 12 extra linking constraints.

Model B is strictly larger and harder for the solver than Model A.

The Exploded Architecture of Model B



Perceived: 7 Variables



Actual: 25 Variables & 18 Constraints

- New Variable Reality: 6 Integer + 18 Auxiliary Binary + 1 Continuous = 25 Total Variables.
- New Constraint Reality: 12 Linking Constraints + 3 Makespan + 3 Memory = 18 Total Constraints.
- The Result: The attempt to reduce variables created a strictly larger and mathematically harder problem.

NotebookLM

The Formulation Scorecard

Criterion	Model A (Binary)	Model B (Integer + Aux)
Primary Variables	18 binary + 1 cont.	6 integer + 18 binary + 1 cont.
Total Constraints	12	18
Formulation Effort	Straightforward matrix	Requires deep linearization insight
Solver Performance	Typically faster, tighter LP relaxation	Typically slower, weaker relaxation

Modern Context: Solvers like Gurobi offer tools like `addGenConstrIndicator()` to handle Model B's indicator functions cleanly in code, but under the hood, the solver is still doing the heavy lifting to linearize the messy math.

NotebookLM

Key Takeaways

1. **Fewer variables ≠ simpler formulation**
   Integer y(i) pushes complexity into constraints. The relationship between variables and constraints matters more than variable count.

2. **The minimax trick is fundamental**
   Introduce T_max, add linking inequalities. This pattern appears in scheduling, load balancing, and resource allocation.

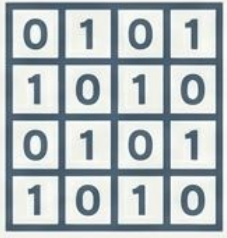
3. **Binary variables suit assignment problems**
   A binary indicator matrix directly supports linear expressions. Solvers exploit this structure.

4. **Model choice affects solver behavior**
   Solvers use special branching and cuts for binary assignment. Integer+auxiliary formulations obscure this structure.

Architectural Truths in Optimization

Fewer != Simpler

Variable count is a deceptive metric. The relationship between a variable's value and the objective constraints matters infinitely more than raw quantity.

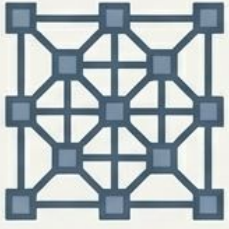


The Power of Binary

Binary variables are the natural, most efficient language for assignment problems. They directly support linear expressions over categories.

The Minimax Standard

Using an auxiliary variable to "push down the ceiling" is a universal, fundamental scheduling trick for load balancing.



Solvers Love Structure

Solvers are optimized to exploit the clean structure of binary assignment matrices. Integer + auxiliary models obscure this underlying structure.

NotebookLM

Exercises for More Practice [Optional]

1. Remove memory constraints from both models. Does the optimal makespan change? Why or why not?
2. Add a constraint: T2 and T3 must be on the same processor. Write it for both models — which is easier?
3. Add a constraint: T1 and T6 must be on different processors. Compare both models.
4. Scale to 12 tasks, 4 processors. Compare solver runtime for Model A vs Model B.
5. *Think ahead: If tasks had precedence constraints (T2 cannot start until T1 finishes), how would the formulation change?*

   Exercise 5 leads to scheduling — coming in a future lectures.

Simulated Annealing

A DIFFERENT APPROACH:
Metaheuristics

Why Leave ILP Behind?

Model A is provably optimal and fast on small instances. But three forces may push us toward a different tool:

Scale

6 tasks × 3 processors = 18 binaries. 1,000 tasks × 20 processors = 20,000 binaries. Branch-and-bound explores a combinatorial tree — solve time grows super-polynomially in the worst case.

Time Budget

ILP guarantees optimality. Embedded deployments often need a good answer in 1 second, not a perfect one in 1 hour. Some applications re-schedule thousands of times per day.

Constraint Shape

If a new constraint isn't linear — e.g. "tasks T2, T3, T5 together must not co-locate" — the ILP grows ugly with extra indicator variables. A heuristic just runs the check.

Sometimes we may trade guaranteed optimality for speed, scale, and flexibility.

INSIGHT

Remember y(i)? It's Back.

In Model B (ILP)

$y(i) \in \{1, \ldots, m\}$

Mathematically elegant.

But forced the solver to linearize indicators — demanded 18 auxiliary binaries and 12 linking constraints.

The integer-vector encoding did not survive contact with linear programming.

In Simulated Annealing

$y = [1, 2, 3, 1, 3, 2]$

The state is just a 6-vector of integers.

No z-variables. No linking constraints. No LP relaxation.

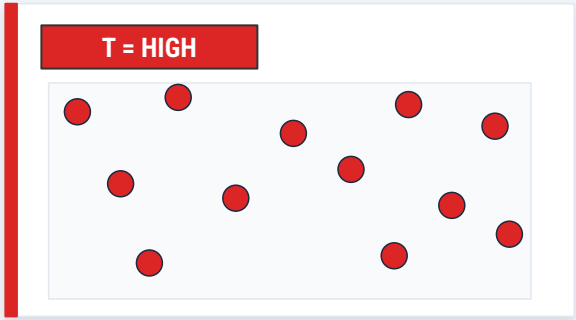
A candidate solution is just an array. An objective is just a function you call on it.

The encoding that a linear solver hated is now the encoding a metaheuristic loves.

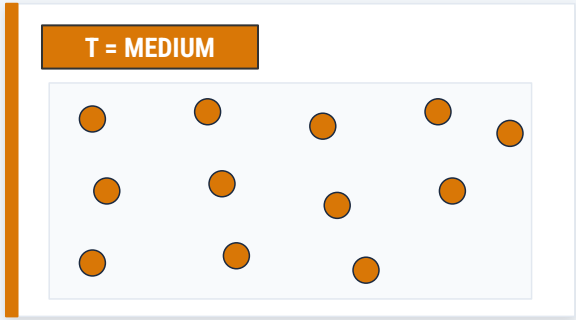
Where the Idea Comes From: Metallurgy

Simulated annealing borrows its mechanics — and its math — from the physics of crystal formation.

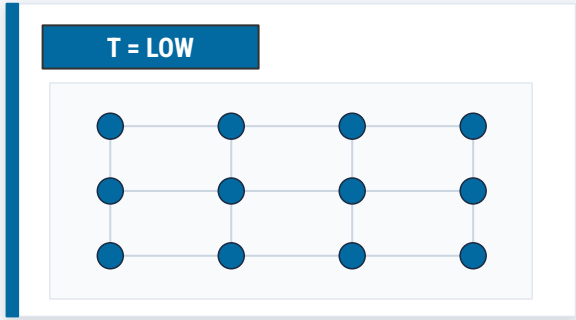
**Kirkpatrick, Gelatt & Vecchi, Science 1983 — the paper that crossed statistical mechanics into combinatorial optimization.*



Atoms move freely; every configuration is accessible.



Clusters form. Local reorganization still happens.



Atoms lock into the minimum-energy crystalline lattice.

cooling → **Slow cooling:** reaches the ground state. **Fast quenching:** freezes in defects — a local minimum.

Physical Annealing	Optimization (Simulated Annealing)
Atomic configuration	Candidate solution y
Energy E of the configuration	Objective $f(y)$ — e.g., makespan
Temperature T	Control parameter T (same symbol, same role)
Boltzmann factor* $\exp(-\Delta E / k_B T)$	**Acceptance probability $\exp(-\Delta E / T)$**

***dimensionless quantity determining the relative probability of a system occupying a specific energy state (ΔE) at a given absolute temperature (T)**

OVERVIEW

Simulated Annealing Approach

1

Initial State

Some initial (e.g., random) solution: state y.

2

Neighborhood

Given the current state y, what states y' can you reach in one step? A small, local perturbation.

3

Acceptance Rule

When you reach a neighbor, do you move there? Downhill: always. Uphill: sometimes — with a probability that depends on how bad the uphill is.

4

Cooling Schedule

How does the willingness to accept uphill moves decay over time? Start exploratory, end greedy.

Random Initial State and The Neighborhood: One Reassignment

Let this be the initial random state: **y = [1, 2, 3, 1, 3, 2]**

The simplest move to get a neighbor state

[1, 2, 3, 1, 3, 2] Before → y

↓ ***pick task 3, reassign from P3 to P1***

[1, 2, 1, 1, 3, 2] After → y'

One entry changed. Everything else preserved.

Pseudocode

```
i  ← random task in {1..n}
j  ← random proc in {1..m}\{y(i)}
y' ← y with y(i) replaced by j
```

Other moves

- Swap: two tasks exchange processors
- Multi-move: reassign k tasks at once

Small, local perturbations keep the math trivial and mix the search space smoothly. Bigger moves converge faster but can disrupt good structure.

The Metropolis Rule: Accepting Uphill Moves

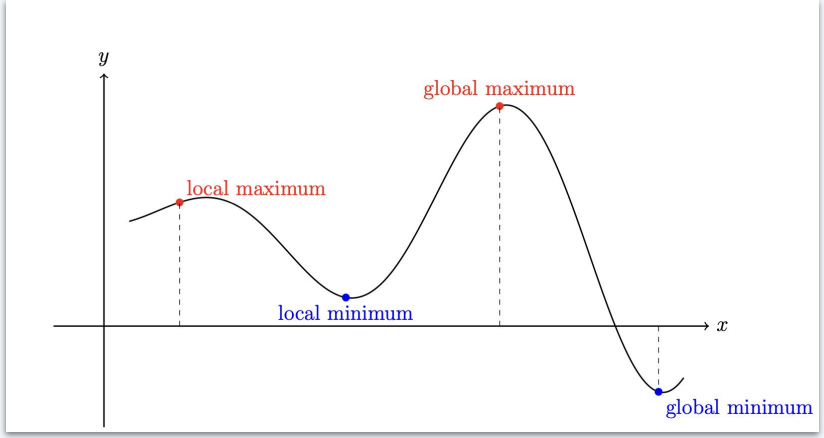
ΔE = makespan(y') − makespan(y)

If $\Delta E \leq 0$ → accept

Downhill moves are always taken. The neighbor is at least as good.

If $\Delta E > 0$ → accept with $P = \exp(-\Delta E / T)$

Uphill moves are accepted probabilistically. Worse moves are rarer than mildly-worse ones. High T makes uphill cheap; low T makes it nearly forbidden.



Why accept uphill at all?

Pure greedy search (only accept downhill) gets trapped in local minima.

Accepting occasional uphill moves lets SA climb out of a valley to discover a deeper one.

This is why SA ≠ hill-climbing.

The exponential form isn't arbitrary — it comes from statistical mechanics and guarantees asymptotic convergence to the global optimum under a logarithmic cooling schedule.

Cooling: Lowering the Temperature

Geometric schedule

$$T_{k+1} = \alpha \cdot T_k$$

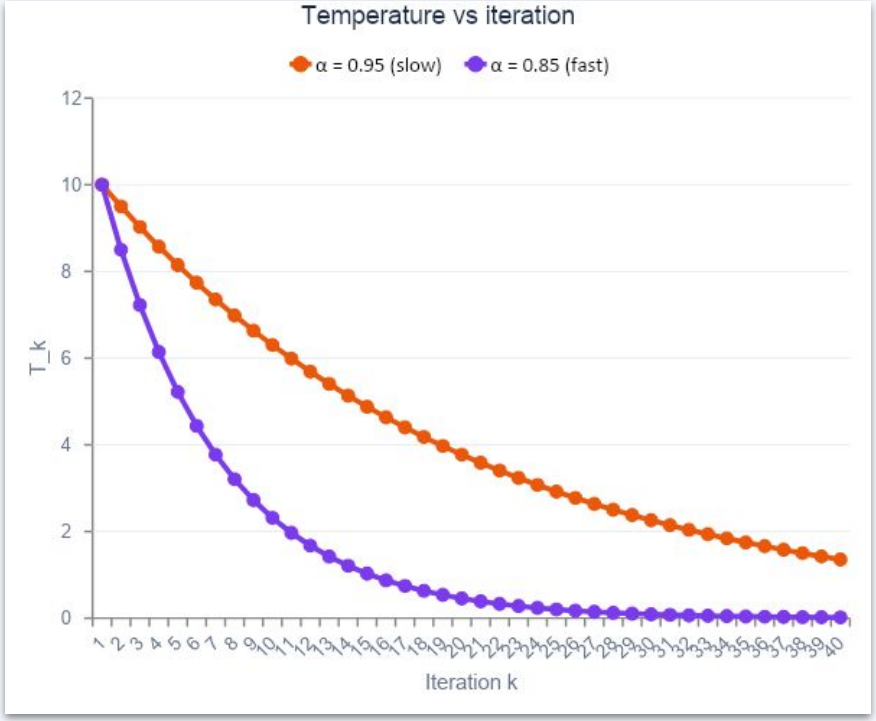
E.g., $\alpha \in [0.85, 0.99]$

Two knobs

- T_0 (initial temperature) — pick so ~80% of uphill moves are accepted at the start.
- α (cooling rate) — slower cooling = better quality, more runtime.

Stopping condition(s)

$T < T_{min}$, iteration budget exhausted, or no improvement in N iterations.



CHOICE

Constraints: Two Strategies

ILP enforces feasibility by construction — the solver simply never considers infeasible assignments. SA has to make a choice.

A · Reject Infeasible

if violates(y') → sample another neighbor

- **Pro**: simple, zero tuning.
- **Pro**: every accepted state is feasible.
- **Con**: if feasible region is disconnected or has narrow corridors, SA can get stuck.
- **Con**: wastes neighbor samples on rejections.

B · Penalize in Objective

$$E(y) = \text{makespan}(y) + \lambda \cdot \text{violation}(y)$$

- **Pro**: SA can cross infeasible territory to reach better feasible regions.
- **Pro**: no wasted samples.
- **Con**: the final state may be infeasible if λ is too small.
- **Con**: λ itself becomes a tuning parameter (often annealed too).

Example

Hand-Trace: The Same 6×3 Problem

Problem data (our ILP example)

Execution times t(i,j)

	P1	P2	P3
T1	3	7	5
T2	8	2	6
T3	6	3	4
T4	4	5	9
T5	5	8	3
T6	7	4	8

Memory

Task	mem
T1	2
T2	3
T3	2
T4	4
T5	1
T6	3

Mem. Capacities: P1 ≤ 8, P2 ≤ 7, P3 ≤ 6

SA parameters: T_0 = 10, α = 0.9, reject infeasible.

Initial state $y^{(0)}$

[1, 2, 1, 3, 1, 2]

i.e. T1→P1, T2→P2, T3→P1, T4→P3, T5→P1, T6→P2

Processor	Tasks	Exec load	Memory
P1	T1, T3, T5	3+6+5 = 14	2+2+1 = 5 ≤ 8 ✓
P2	T2, T6	2+4 = 6	3+3 = 6 ≤ 7 ✓
P3	T4	9	4 ≤ 6 ✓

Makespan($y^{(0)}$):

14

Sum of best-case times:
3+2+3+4+3+4=19

$$\text{Makespan} \geq \frac{19}{3} \approx 6.33$$

Lower bound (best per task): 7
Gap to optimum: 7

$$\lceil 6.33 \rceil = 7$$

TRACE

Hand-Trace: Iterations 1–3

Iter	Move	Proposed y'	Check	Makespan	ΔE	Decision
1	T3: P1 → P2	[1,2,**2**,3,1,2]	**P2 mem: 3+2+3 = 8 > 7**	—	—	**REJECT (infeasible)**
2	T3: P1 → P3	[1,2,**3**,3,1,2]	**all mem OK; loads 8 / 6 / 13**	**13**	**−1**	**ACCEPT (downhill)**
3	T4: P3 → P1	[1,2,3,**1**,1,2]	**all mem OK; loads 12 / 6 / 4**	**12**	**−1**	**ACCEPT (downhill)**

What we've seen so far: The first proposed move violates P2's tight memory budget and is discarded — no objective evaluation needed. The next two proposals are both downhill, and both are accepted unconditionally. No uphill has been tried yet.

Current state: y = [1, 2, 3, 1, 1, 2], makespan = 12, $T \approx 10 \cdot 0.9^3 = 7.29$.

TRACE Hand-Trace: The Uphill Escape

Current state: y = [1, 2, 3, 1, 1, 2],

Iteration 4 — uphill!

Propose T5: P1 → P2

y' = [1, 2, 3, 1, 2, 2]

P1 mem: 2+4 = 6 ≤ 8 ✓

P2 mem: 3+1+3 = 7 ≤ 7 ✓

P3 mem: 2 ≤ 6 ✓

Loads: 7 / 14 / 4 → makespan = 14

ΔE = 14 − 12 = +2 (uphill)

P = exp(−2 / 7.29) ≈ 0.76. Draw r = 0.62 < 0.76 → ACCEPT

Iteration 5 — the payoff

Propose T5: P2 → P3

y' = [1, 2, 3, 1, 3, 2]

P1 mem: 2+4 = 6 ≤ 8 ✓

P2 mem: 3+3 = 6 ≤ 7 ✓

P3 mem: 2+1 = 3 ≤ 6 ✓

Loads: 7 / 6 / 7 → makespan = 7

ΔE = 7 − 14 = −7 (big downhill)

ACCEPT → makespan = 7 (matches the ILP optimum!)

Without the uphill acceptance in Iteration 4, we could not have reached the optimum in Iteration 5. This is the why SA is good.

COMPARE

ILP vs SA: The Scorecard

Criterion	ILP (Model A)	SA
Optimality	Guaranteed by branch-and-bound.	No guarantee. Often near-optimal in practice; depends on tuning and seed.
Runtime — small instance	Milliseconds.	Milliseconds.
Runtime — large instance	Minutes to hours, possibly runs out of memory.	Seconds — independent of problem structure.
Constraint flexibility	Linear constraints only. Non-linear requires big-M and auxiliary variables.	Any constraint expressible as a check — linear or not.
Solution variance across runs	Zero. Deterministic, reproducible.	Non-zero. Report best-of-N or mean ± std over multiple seeds.
Up-front effort	Formulate carefully once; solver does the rest.	Tune T_0, α, stopping criterion, and possibly λ.

Complementary, not competing. ILP for small/medium instances where optimality matters. SA for large instances, tight time budgets, or problems where constraints get messy for LP.

Exercises for More Practice [Optional]

Try these before the next lecture

1. Implement SA with reject-infeasible. Run it 30 times on the small instance. Histogram the best makespans. How often do you hit 7?
2. Switch to the penalty method. How does the choice of λ affect (a) final solution quality and (b) feasibility rate at the end of the run?
3. Add a swap move (exchange two tasks' processors) to the neighborhood. Does mixing two move types accelerate convergence?
4. Try $T_0 = 0.01$ and $\alpha = 0.5$. What happens? What does this reveal about the roles of T_0 and α?
5. *Preview: hybrid methods. You can use an ILP relaxation to seed SA's initial state. If you try this, do you get the best of both worlds?*